GPIO Driver: Simple LED Game

Overview

This project implements a bare-metal General Purpose Input/Output (GPIO) driver to create a simple interactive LED reflex game on the STM32. It serves as the foundation for physical hardware interaction, managing standard Push-Pull outputs for visual feedback and Input polling for player actions.

Objectives

- **Develop a custom GPIO driver to abstract AHB1 bus memory mapping and register manipulation.**
- **Configure pins dynamically for INPUT (Push Button) and OUTPUT (LED).**
- **Implement software debouncing to ensure clean and accurate button presses.**
- **Build a basic state machine to handle game logic and adjust difficulty dynamically.**

Components Used

- **STM32 Nucleo Board (STM32F401RE / F446RE)**
- **Internal Green LED (PA5)**
- **Internal User Button (PC13)**
- **STM32CubeIDE**

Working Principle

The GPIO driver directly writes to and reads from the MODER, OTYPER, OSPEEDR, PUPDR, ODR, and IDR registers. The game relies on a continuous polling loop: the onboard LED blinks at a specific interval. The player must press the user button exactly when the LED is ON. The driver reads the IDR (Input Data Register) of Port C to detect the button press, and evaluates the ODR (Output Data Register) of Port A to determine if the player won or lost the round.

System Architecture

- **GPIO Driver: Hides bitwise shifting complexity through a GPIO_Handle_t struct.**
- **Input Polling Logic: Continuously monitors the user button state (active LOW on Nucleo boards).**
- **Game Logic:**
    - **Win: Pressing the button while the LED is ON increases the game speed (decreasing the blink delay).**
    - **Loss: Pressing the button while the LED is OFF resets the game to the slowest speed.**

Implementation Flow

1. **Initialize the GPIO clocks for Port A and Port C via RCC registers.**
2. **Configure PA5 as a Push-Pull output for the LED.**
3. **Configure PC13 as an Input for the User Button.**
4. **Enter the infinite while(1) game loop to toggle the LED.**

5. **Poll the button state, apply a short delay for debouncing, and evaluate the win/loss condition.**

Key Observations

- **Mechanical Bounce: Physical buttons generate noisy signals when pressed. Adding a small software delay (e.g., 50ms) after detecting the initial press was critical to prevent the microcontroller from registering multiple false inputs.**
- **Pin Abstraction: The custom driver successfully hid the complex bit-masking required to read specific input pins and toggle output pins cleanly.**

Results

Successfully achieved a stable, playable reflex game that responds instantly to user input and dynamically scales in difficulty.

Key Learnings

- **Mastering GPIO memory offsets and register masking for both Input and Output configurations without high-level HAL libraries.**
- **Understanding the necessity of software debouncing when interfacing with mechanical switches.**

💻 **Source Code: https://github.com/adlernunez/Adler-Entri/tree/master/ARM%20F401xx/FINAL%20PROJECT/STM32F446RE**

Application Layer (main.c snippet)

```
#include "stm32f401re_gpio_driver.h"


// Crude delay function for game pacing and debouncing
void delay_ms(uint32_t ms) {
   for(uint32_t i = 0; i < ms * 4000; i++);
}


int main(void) {
   // 1. Initialize the Onboard Green LED (PA5)
   GPIO_handle_t led_green = {0};
   led_green.pGPIOx = GPIOA;
   led_green.GPIO_PinConfig.GPIO_PinNumber = GPIO_PIN_NO_5;
   led_green.GPIO_PinConfig.GPIO_PinMode = GPIO_MODE_OUT;
   led_green.GPIO_PinConfig.GPIO_PinSpeed = GPIO_SPEED_FAST;
   led_green.GPIO_PinConfig.GPIO_PinOPType = GPIO_PUSHPULL;
   led_green.GPIO_PinConfig.GPIO_PinPuPdControl = GPIO_NO_PUPD;
```

```
  GPIO_Init(&led_green);

  // 2. Initialize the Onboard User Button (PC13)
  GPIO_handle_t user_btn = {0};
  user_btn.pGPIOx = GPIOC;
  user_btn.GPIO_PinConfig.GPIO_PinNumber = GPIO_PIN_NO_13;
  user_btn.GPIO_PinConfig.GPIO_PinMode = GPIO_MODE_IN;
  user_btn.GPIO_PinConfig.GPIO_PinSpeed = GPIO_SPEED_FAST;
  user_btn.GPIO_PinConfig.GPIO_PinPuPdControl = GPIO_NO_PUPD; // Nucleo board has external pull-up
  GPIO_Init(&user_btn);

  uint32_t game_speed = 500; // Initial LED toggle delay (ms)

  while(1) {
    // Toggle the LED
    GPIO_ToggleOutputPin(GPIOA, GPIO_PIN_NO_5);
    delay_ms(game_speed);

    // Read Button State (Nucleo PC13 is Active LOW)
    if(GPIO_ReadFromInputPin(GPIOC, GPIO_PIN_NO_13) == 0) {

      delay_ms(50); // Software Debounce

      // Confirm button is still pressed
      if(GPIO_ReadFromInputPin(GPIOC, GPIO_PIN_NO_13) == 0) {

        // Read current state of LED to check win condition
        uint8_t led_state = GPIO_ReadFromInputPin(GPIOA, GPIO_PIN_NO_5);

        if(led_state == 1) {
```

```
            // WIN: LED was ON. Speed up the game!
            if(game_speed > 100) game_speed -= 50;
        } else {
            // LOSE: LED was OFF. Reset speed.
            game_speed = 500;
        }

        // Wait for player to release the button before continuing
        while(GPIO_ReadFromInputPin(GPIOC, GPIO_PIN_NO_13) == 0);
        delay_ms(50); // Debounce release
      }
    }
  }
}
```

DEMO: https://drive.google.com/drive/folders/1eXq7sDkj5dFtnOcIj5tX9Nme-Wcqtehi?usp=sharing